

ADSO Training 3

Scripts d'automatització de
tasques

Índex

1. Introducció	2
1.1. Objectius	2
2. Com començar	3
2.1. Comproveu la versió de python instal·lada	3
2.2. Integrated Development Environment (IDE): spyder3	3
2.3. Propietats d'un bon script	3
3. Script 1: Usuaris innecessaris	4
3.1. El script en Bash	5
3.2. Script en Python	6
4. Detecció de usuaris inactius	6
4.1. El script en Bash	7
4.2. Script en Python	7
5. Script per a la gestió de l'espai en disc	8
5.1. El script en Bash	9
5.2. Script en Python	9
6. Informació d'usuaris	10
6.1. El script en Bash	10
6.2. Script en Python	11
7. Estadístiques de login y processos	11
7.1. El script en Bash	12
7.2. Script en Python	12
8. Estadístiques de comunicació	12
8.1. El script en Bash	13
8.2. Script en Python	13
9. Activitat dels usuaris	14
9.1. El script en Bash	14
9.2. Script en Python	15
10. Referències Bibliogràfiques	15

1. Introducció

Normalment les tasques d'administració han de repetir-se una vegada i una altra, motiu pel qual l'administrador ha d'escriure de nou les comandes i en ocasions canviant només algun paràmetre d'entrada. Fer aquestes tasques manualment no només implica una inversió considerable de temps sinó que exposa el sistema a errors quan es repeteix una comanda de forma equivocada. L'automatització d'aquestes tasques mitjançant llenguatges d'script millora l'eficiència del sistema ja que aquestes es realitzen sense la intervenció de l'administrador. També augmenta la fiabilitat perquè les comandes es repeteixen de la mateixa forma cada vegada i a més a més permet garantir la regularitat en la seva execució perquè aquestes tasques es poden programar fàcilment perquè s'executin periòdicament.

Encara que l'automatització es podria fer en qualsevol llenguatge de programació existeixen llenguatges, coneguts com llenguatges *d'scripting*. Aquests llenguatges permeten combinar fàcilment expressions del propi llenguatge d'script amb comandes del sistema, i també faciliten la manipulació de fitxers de text, llistes, i el recorregut i tractament de directoris, i altres tasques útils per a l'administració. Existeixen múltiples llenguatges *d'scripting*: els associats al shell (com *Bash* o *C shell*) i altres amb funcionalitats més esteses com *Perl* o *Python*.

1.1. Objectius

Aprendre a automatitzar algunes tasques comunes d'administració de sistemes fent ús de llenguatges d'script com el Bash i el Python. Tasques a automatitzar (en Python i Bash):

Script 1: Determinar quins usuaris del fitxer `/etc/passwd` són invàlids.

Script 2: Gestió de l'espai en disc utilitzat per cada usuari del sistema.

Script 3. Informació d'usuaris

Script 4. Estadístiques de login y processos

Script 5. Estadístiques de comunicació

Script 6. Activitat dels usuaris

2. Com començar

Aprendre la programació de shell-scripts amb Bash [1]

Analitzar les construccions bàsics del llenguatge Python [4]

2.1. Comproveu la versió de python instal·lada

Tots els usuaris del sistema han de poder executar les dues versions:

#python (executarà la versió inferior)

#python3 (executarà la versió actualitzada)

```
root@davidP (dj. d'oct. 23):~# python -version
bash: python: no s'ha trobat l'ordre
root@davidP (dj. d'oct. 23):~# python3 -version
Unknown option: -e
usage: python3 [option] ... [-c cmd | -m mod | file | -] [arg] ...
Try `python -h' for more information.
```

2.2. Integrated Development Environment (IDE): spyder3

Instal·leu un Integrated Development Environment (IDE): spyder3 (busca la web oficial). Tots els usuaris del sistema han de poder utilitzar aquesta eina

Per a que serveix? Que característiques te? Descriu el procés d'instal·lació

Spyder (que es la abreviatura de Scientific PYthon Development EnviRonment) es un Entorno de Desarrollo Integrado (IDE) de código abierto. Está diseñado específicamente para la computación científica, el análisis de datos y la ingeniería, usando el lenguaje Python.

Las características principales que lo hacen popular para la ciencia de datos son:

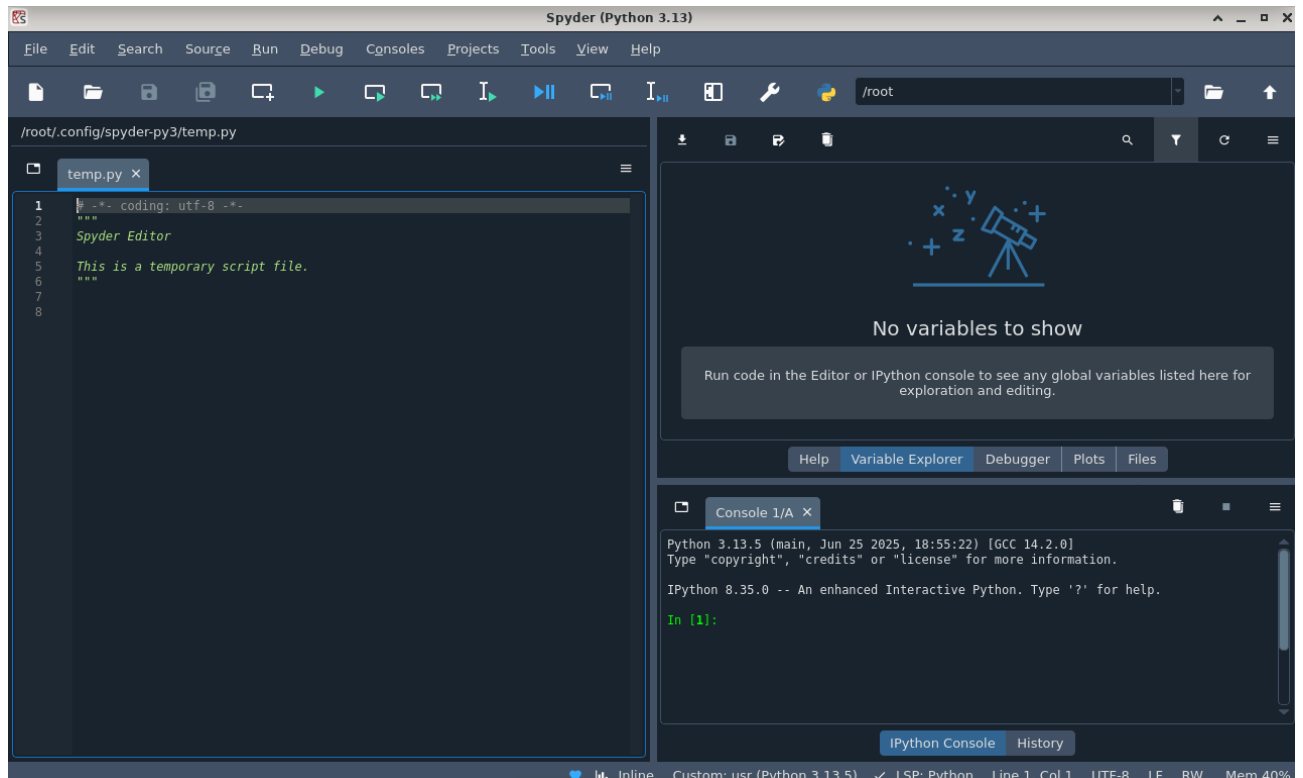
- Editor de código avanzado: Incluye resaltado de sintaxis, autocompletado y análisis de errores en tiempo real.
- Consola Python interactiva: Permite ejecutar código línea por línea o por bloques y ver los resultados al instante, lo cual es muy útil para explorar datos.
- Explorador de variables: Esta es quizás su característica más destacada. Permite inspeccionar gráficamente todas las variables, listas y dataframes (como tablas de Excel) que están en la memoria de tu programa.
- Depurador (Debugger): Una herramienta integrada para encontrar y corregir errores (bugs) en el código paso a paso.
- Paneles integrados: Tiene paneles especiales para ver gráficos y gráficas, explorar archivos y leer la documentación sin salir del programa.

```

root@davidP (dl. de nov. 10):~# sudo apt install spyder
spyder ja està en la versió més recent (6.0.5+ds-6).
El paquets següents s'han instal·lat automàticament i ja no són necessaris:
bsdmainutils      libflac8          libmpdec3         libpython3.7-minimal  libtirpc-dev      python3.7-minimal
libaom0           libicu63          libnsl-dev        libpython3.7-stdlib  libx265-165       python3.9
libapt-pkg6.0     libicu67          libopenexr23      libpython3.9-minimal  libx265-192       python3.9-minimal
libcbor0          libilmbase23      libperl5.28       libpython3.9-stdlib  linux-image-4.19.0-6-amd64  ruby2.7
libdav1d4         libisc-export1105 libperl5.32       libreadline7         perl-modules-5.28
libdns-export1110 libmpdec2         libprocps8        libruby2.5           perl-modules-5.32
Empreu «sudo apt autoremove» per a suprimir-los.

Resum:
S'està actualitzant: 0, s'està instal·lant: 0, S'està suprimint: 0, no s'està actualitzant: 48

```



2.3. Propietats d'un bon script

- 1) Un script ha d'executar-se sense errors.
- 2) Ha de realitzar la tasca per a la qual està pensat.
- 3) La lògica del programa ha d'estar clarament definida.
- 4) Un script no ha de fer treball innecessari.
- 5) Els scripts han de ser reutilitzables.

3. Script 1: Usuaris innecessaris

Es demana fer un script que determini quins usuaris del fitxer **/etc/passwd** són invàlids. Un usuari invàlid és aquell que existeix en el fitxer de **passwd** però que en canvi no té cap presència en el sistema (és a dir, que no té cap fitxer). També, hi ha usuaris que no tenen cap fitxer, però que serveixen per executar *daemons* del sistema. Afegiu una

opció per declarar vàlids als usuaris que tenen algun procés en execució (flag -p).

Exemple de la sortida:

```
$/BadUsers.sh
daemon
bin
sys
sync
games
lp
mail
news
aduran
alvarez
proxy
backup

$/BadUsers.sh -p
bin
sync
games
lp
news
aduran
alvarez
proxy
backup
```

3.1. El script en Bash

Teniu el un script fet en Bash. Completeu els espais buits amb les comandes apropiades.

```
None
#!/bin/bash
# Script per trobar usuaris invàlids (sense fitxers)
# Font: Pràctica Training 3, Apartat 3.1

p=0
usage="Usage: BadUser.sh [-p]"
```

```

# detecció de opcions d'entrada: només son vàlids: sense paràmetres i -p
if [ $# -ne 0 ]; then
    if [ $# -eq 1 ]; then
        if [ $1 == "-p" ]; then
            p=1
        else
            echo $usage; exit 1
        fi
    else
        echo $usage; exit 1
    fi
fi

# Comanda per llegir el fitxer de password i agafar el camp de nom de l'usuari
for user in `cat /etc/passwd | cut -d: -f1`; do

    # Obtenim el directori 'home' de l'usuari
    home=`cat /etc/passwd | grep "^$user\>" | cut -d: -f6`

    if [ -d $home ]; then
        # Comprovem si té fitxers al seu 'home'
        num_fich=`find $home -type f -user $user 2>/dev/null | wc -l`
    else
        num_fich=0
    fi

    # Si no té fitxers
    if [ $num_fich -eq 0 ]; then
        if [ $p -eq 1 ]; then
            # Si s'ha activat -p, comprovem si té processos

            # Comanda per detectar si l'usuari té processos en execució
            # --no-headers és per evitar comptar la línia de capçalera de 'ps'
            user_proc=`ps -u $user --no-headers | wc -l`

            if [ $user_proc -eq 0 ]; then
                echo "$user"
            fi
        fi
    fi
done

```

```
        fi
    else
        # Si -p no està actiu, el mostrem directament
        echo "$user"
    fi
fi
fi
done
```

Què vol dir exactament aquesta comanda:

```
`cat /etc/passwd | grep "^$user\>" | cut -d: -f6`?
```

`cat /etc/passwd`: Llegeix el contingut del fitxer d'usuaris.

`| grep "^$user\>":` Filtra les línies. Aquesta és la part clau:

Cerca la variable `$user`.

`^` assegura que el nom de l'usuari estigui al principi de la línia.

`\>` assegura que sigui una paraula completa (evita que si `$user` és "usu", trobi "usuari1").

`| cut -d: -f6`: Talla la línia resultant (que ja és només la de l'usuari) usant `:` com a separador i agafa el sisè camp (`-f6`), que correspon al directori "home" de l'usuari.

Quina diferencia hi ha amb la comanda:

```
`cat /etc/passwd | grep "$user" | cut -d: -f6`?
```

La comanda sense `^` i `\>` (és a dir, `grep "$user"`) faria una cerca de "subcadena".

Exemple: Si `$user` és "alex", `grep "$user"` trobaria línies que continguin "alex", "alexia", "ppalex", etc. Si "ppalex" apareix abans que "alex", la comanda podria agafar el directori "home" de "ppalex" per error.

La primera comanda (`grep "^$user\>"`), en canvi, és exacta i només trobarà la línia que comença exactament amb "alex:", evitant errors.

Mostra la sortida de l'execució del script


```
root@davidP (dl. de nov. 10):~# ./BadUsers.sh
daemon
bin
sys
sync
games
lp
mail
news
uucp
proxy
www-data
backup
list
irc
gnats
nobody
_apt
systemd-network
systemd-resolve
messagebus
aso
systemd-coredump
dnsmasq
cups-pk-helper
pulse
speech-dispatcher
usbmux
saned
rtkit
geoclue
```

3.2. Script en Python

Feu el mateix script amb llenguatge Python

None

```
#!/usr/bin/env python3
#
# Script per trobar usuaris invàlids (sense fitxers)
# Rèplica de l'script Bash de l'apartat 3.1
#
import sys
import os
```

```

import subprocess

def get_user_processes(user: str) -> int:
    """
    Comprova quants processos té un usuari en execució.
    Executa: ps -u $user --no-headers | wc -l
    """
    command = f"ps -u {user} --no-headers | wc -l"
    try:
        result = subprocess.run(command, shell=True, capture_output=True,
text=True, check=True)
        # Convertim la sortida (un text) a un enter
        return int(result.stdout.strip())
    except (subprocess.CalledProcessError, ValueError):
        # Si la comanda falla o la sortida no és un número, assumim 0
        return 0

def get_user_files(user: str, home_dir: str) -> int:
    """
    Comprova quants fitxers té un usuari al seu directori home.
    Executa: find $home -type f -user $user 2>/dev/null | wc -l
    """
    if not os.path.isdir(home_dir):
        # Si el directori home no existeix, no té fitxers
        return 0

    # Afegim 2>/dev/null per suprimir errors de "permís denegat"
    command = f"find {home_dir} -type f -user {user} 2>/dev/null | wc -l"
    try:
        result = subprocess.run(command, shell=True, capture_output=True,
text=True, check=True)
        return int(result.stdout.strip())
    except (subprocess.CalledProcessError, ValueError):
        return 0

def main():
    # Comprovem si s'ha passat el flag -p

```

```

check_processes = False
if len(sys.argv) > 1:
    if sys.argv[1] == "-p":
        check_processes = True
    else:
        # Mostrem l'ús correcte si el paràmetre no és -p
        print(f"Usage: {sys.argv[0]} [-p]", file=sys.stderr)
        sys.exit(1)

try:
    # Obrim el fitxer /etc/passwd per llegir-lo
    with open("/etc/passwd", "r") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith('#'):
                continue

            try:
                # Separem la línia pel caràcter ':'
                parts = line.split(':')
                username = parts[0]
                home_dir = parts[5]
            except (IndexError, ValueError):
                # Ignorem línies mal formades
                continue

            # Comprovem el número de fitxers
            num_fich = get_user_files(username, home_dir)

            # Si no té fitxers, és un candidat
            if num_fich == 0:
                if check_processes:
                    # Si s'ha activat -p, mirem els processos
                    user_proc = get_user_processes(username)
                    if user_proc == 0:
                        print(username) # Només si no té fitxers NI

```

processos

```
        else:
            # Si -p no està actiu, el mostrem directament
            print(username) # No té fitxers

    except FileNotFoundError:
        print("Error: No s'ha pogut obrir /etc/passwd", file=sys.stderr)
        sys.exit(1)
    except Exception as e:
        print(f"Ha ocorregut un error inesperat: {e}", file=sys.stderr)
        sys.exit(1)

if __name__ == "__main__":
    main()
```

Mostra la sortida de l'execució del script

```
root@davidP (dl. de nov. 10):~# nano BadUsers.py
root@davidP (dl. de nov. 10):~# chmod +x BadUsers.py
root@davidP (dl. de nov. 10):~# python3 BadUsers.py
daemon
bin
sys
sync
games
lp
mail
news
uucp
proxy
www-data
backup
list
irc
gnats
nobody
_appt
systemd-network
systemd-resolve
messagebus
aso
systemd-coredump
dnsmasq
cups-pk-helper
pulse
speech-dispatcher
usbmux
saned
rtkit
geoclue
root@davidP (dl. de nov. 10):~# █
```

4. Detecció de usuaris inactius

Ara esteneu aquest script per detectar usuaris *inactius*. Es defineix com inactius als que no executen cap procés (veure comanda **ps** al punt anterior), que fa molt de temps que no han fet login (ver comandes **finger**, **last** i **lastlog**), i que fa molt de temps que no han modificat cap dels seus fitxers (veure opcions *time* a la comanda **find**). El període d'inactivitat s'indicarà a través d'un paràmetre:

```
$/BadUsers.sh -t 2d (indica 2 dies)
```

```
alvarez
```

```
aduran
```

```
xavim
```

```
marcg
```

```
$/BadUsers.pl -t 4m (indica 4 mesos)
```

```
xavim
```

```
marcg
```

4.1. El script en Bash

Feu el script amb Bash

None

```
#!/bin/bash

usage="Usage: BadUser.sh [-t <duration>]"

# Verifica que s'hagi introduït un paràmetre.
if [ "$#" -ne 2 ] || [ "$1" != "-t" ]; then
    echo $usage
    exit 1
fi

duration=$2

# Obté la data de modificació màxima basada en el paràmetre introduït.
case ${duration: -1} in
    "d") max_age=$(( ${duration%?} ));;
    "m") max_age=$(( ${duration%?} * 30 ));;
    *) echo "Durada no reconeguda. Utilitzeu 'd' per dies i 'm' per mesos.";
exit 1;;
esac

for user in $(cut -d: -f1 /etc/passwd); do
    home=$(getent passwd "$user" | cut -d: -f6)
```

```

has_processes=$(ps -ef | grep "^$user " | wc -l)
last_login=$(lastlog -u "$user" | tail -1 | awk '{print $1, $2, $3, $4,
$5}')
last_login_seconds=$(date -d "$last_login" +%s 2> /dev/null)
current_seconds=$(date +%s)
days_since_last_login=$((current_seconds - last_login_seconds) / 86400)

if [ -d "$home" ] && [ "$has_processes" -eq 0 ] && [
"$days_since_last_login" -gt "$max_age" ]; then
    has_recent_files=$(find "$home" -type f -user "$user" -mtime
-"$max_age" 2> /dev/null | wc -l)
    if [ "$has_recent_files" -eq 0 ]; then
        echo "$user"
    fi
fi
done

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# ./BadUsers.sh -t 3m
daemon
bin
sys
sync
games
mail
proxy
backup
systemd-network
systemd-resolve
systemd-coredump
saned

```

4.2. Script en Python

Feu el mateix script amb llenguatge Python

None

```
import subprocess
import datetime
import re
import sys

def get_max_age(duration):
    unit = duration[-1]
    value = int(duration[:-1])
    if unit == 'd':
        return value
    elif unit == 'm':
        return value * 30
    else:
        raise ValueError("Durada no reconeguda. Utilitzeu 'd' per dies i 'm'
per mesos.")

def get_users():
    cmd = "cut -d: -f1 /etc/passwd"
    return subprocess.getoutput(cmd).splitlines()

def get_home_directory(user):
    cmd = f"getent passwd {user} | cut -d: -f6"
    return subprocess.getoutput(cmd)

def user_has_processes(user):
    cmd = f"ps -ef | grep ^{user} | wc -l"
    return int(subprocess.getoutput(cmd)) > 0

def days_since_last_login(user):
    cmd = f"lastlog -u {user}"
    output = subprocess.getoutput(cmd).splitlines()[-1]
    match = re.search(r"\w{3} \w{3} [ \d]{2} [\d:]{5} [\d]{4}", output)
    if match:
        last_login_str = match.group(0)
        last_login_date = datetime.datetime.strptime(last_login_str, "%a %b %d
%H:%M:%S %Y")
        now = datetime.datetime.now()
```



```

        return (now - last_login_date).days
    return float('inf')

def has_recent_files(user, home, max_age):
    cmd = f"find {home} -type f -user {user} -mtime -{max_age} 2>/dev/null | wc
-l"
    return int(subprocess.getoutput(cmd)) > 0

if __name__ == "__main__":
    if len(sys.argv) != 3 or sys.argv[1] != "-t":
        print("Usage: BadUser.py [-t <duration>]")
        sys.exit(1)

    duration = sys.argv[2]
    max_age = get_max_age(duration)

    for user in get_users():
        home = get_home_directory(user)
        if (not user_has_processes(user) and
            days_since_last_login(user) > max_age and
            not has_recent_files(user, home, max_age)):
            print(user)

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# python3 BadUsers.py -t 2m
daemon
bin
sys
sync
games
mail
proxy
backup
systemd-network
systemd-resolve
systemd-coredump
saned

```

5. Script per a la gestió de l'espai en disc

S'ha de fer un script que calculi l'espai en disc utilitzat per cada usuari del sistema. Si sobrepassa un espai determinat que es passa com paràmetre, s'haurà d'escriure un missatge al *.profile* del usuari en qüestió per informar-li que ha d'esborrar/comprimir alguns dels seus fitxers.

Concretament, la sintaxi del programa ha de ser la següent:

\$ ocupacio.sh max_permès

Per exemple:

```
$ ./ocupacio.sh 600M
root      567 MB
alvarez   128 KB
aduran    120 MB
```

Després esteneu el script per afegir una opció per grups **-g**: Amb aquesta opció l'script ha de retornar l'ocupació total per als usuaris del grup **<grup>**, el total d'ocupació del grup sencer, i posar el missatge als usuaris que sobrepassen el **max_permès**.

Per tant, la sintaxi final del programa haurà de ser:

\$ ocupacio.sh [-g grup] max_permès

Per exemple:

```
$ ./ocupacio.sh -g users 500K
alvarez   128 KB
xavim     23 MB
( ... )
```

NOTA: El missatge que s'ha de posar en el *.profile*, ha de poder ser localitzat i esborrat per l'usuari sense cap tipus de problema. Això vol dir que al costat del missatge s'haurien de posar instruccions per poder-lo esborrar sense cap problema.

5.1. El script en Bash

Feu el script amb Bash

```
None
#!/bin/bash
#
```

```

# Script per a la gestió de l'espai en disc (Apartat 5.1)
# Versió refactoritzada, més robusta i segura.

# --- Etiquetes per a l'avís al .profile ---
PROFILE_START_TAG="#--- AUTO-GENERATED DISK USAGE WARNING (START) ---#"
PROFILE_END_TAG="#--- AUTO-GENERATED DISK USAGE WARNING (END) ---#"

# --- Funció per mostrar l'ús correcte (del teu script original) ---
usage() {
    echo "Ús: $0 [-g grup] max_permès"
    echo "max_permès ha de ser un valor seguit de K, M o G (per exemple, 600M)"
    exit 1
}

# --- Funció per escriure l'avís al .profile (SEGURA) ---
# Aquesta funció esborra l'avís antic i n'escriu un de nou.
write_warning() {
    local directori=$1
    local usuari=$2
    local espai_usat_hr=$3
    local limit_hr=$4
    local profile_file="$directori/.profile"

    # 1. Esborrem l'avís antic, si n'hi ha, per evitar duplicats
    # sed -i '/TAG1/,/TAG2/d' -> esborra les línies entre TAG1 i TAG2
    sed -i "/$PROFILE_START_TAG/,/$PROFILE_END_TAG/d" "$profile_file"
2>/dev/null

    # 2. Afegim el nou avís al final del fitxer
    {
        echo "$PROFILE_START_TAG"
        echo "# ATENCIÓ ($usuari): El teu ús de disc ($espai_usat_hr) supera el
límit ($limit_hr).\"
        echo "# Si us plau, esborra o comprimeix fitxers antics.\"
        echo "# Per esborrar aquest missatge, elimina les línies entre les
etiquetes START i END.\"
        echo \"$PROFILE_END_TAG"
    }

```

```

    } >> "$profile_file"

    # 3. Assegurem que el propietari del .profile segueixi sent l'usuari
    chown "$usuari" "$profile_file" 2>/dev/null
}

# --- Funció per convertir KB a format llegible (GB, MB, KB) ---
# La teva lògica, però en una funció reutilitzable.
format_kb_to_hr() {
    local espai_usat_kb=$1

    if [ "$espai_usat_kb" -ge 1048576 ]; then
        local espai_usat_gb=$(echo "scale=2; $espai_usat_kb/1048576" | bc)
        echo "$espai_usat_gb GB"
    elif [ "$espai_usat_kb" -ge 1024 ]; then
        local espai_usat_mb=$(echo "scale=2; $espai_usat_kb/1024" | bc)
        echo "$espai_usat_mb MB"
    else
        echo "$espai_usat_kb KB"
    fi
}

# --- Funció per comprovar l'espai (del teu script original, millorada) ---
comprova_usuari() {
    local usuari=$1
    local directori=$2
    local espai_usat_kb=$3

    # 1. Formatem la sortida
    local espai_usat_hr=$(format_kb_to_hr $espai_usat_kb)
    echo -e "$usuari\t\t$espai_usat_hr" # -e per interpretar el tabulador

    # 2. Comprova si l'espai utilitzat sobrepassa el límit
    if [ "$espai_usat_kb" -gt "$LIMIT_KB" ]; then
        echo "    -> AVÍS: $usuari supera el límit. S'escriurà a .profile."
        write_warning "$directori" "$usuari" "$espai_usat_hr" "$MAX_PERMIS"
    fi
}

```

```

}

# --- 1. Processar Opcions (del teu script original) ---
if [ "$#" -lt 1 ]; then
    usage
fi
GROUP=""
while getopts 'g:' OPTION; do
    case "$OPTION" in
        g)
            GROUP="$OPTARG"
            ;;
        ?)
            usage
            ;;
    esac
done
shift "$((OPTIND - 1))"
if [ "$#" -ne 1 ]; then
    usage
fi
MAX_PERMIS=$1

# --- 2. Convertir límit a KB (Més robust, sense bc) ---
number=$(echo "$MAX_PERMIS" | sed 's/^[^0-9]*/g')
unit=$(echo "$MAX_PERMIS" | sed 's/[0-9]*/g' | tr '[:lower:]' '[:upper:]') #
Converteix a majúscules

LIMIT_KB=0
case "$unit" in
    K|KB) LIMIT_KB=$((number));;
    M|MB) LIMIT_KB=$((number * 1024));;
    G|GB) LIMIT_KB=$((number * 1024 * 1024));;
    "") LIMIT_KB=$((number));; # Si no hi ha unitat, assumim KB
    *) echo "Unitat '$unit' no reconeguda."; usage;;
esac

```

```

if [ $LIMIT_KB -eq 0 ]; then
    echo "Error: Límit '$MAX_PERMIS' no vàlid."
    usage
fi

# --- 3. Execució Principal ---
total_kb=0
echo "Calculant ús de disc (Límit: $MAX_PERMIS)..."

if [ -n "$GROUP" ]; then
    # --- Mode Grup ---
    membres_grup=$(getent group $GROUP | cut -d: -f4)
    if [ -z "$membres_grup" ]; then
        echo "Error: Grup '$GROUP' no trobat o sense membres."
        exit 1
    fi

    for usuari in ${membres_grup//,/ }; do
        directori_usuari=$(getent passwd $usuari | cut -d: -f6)
        if [ -d "$directori_usuari" ]; then
            # CORRECCIÓ CLAU: Utilitzem 'du -sk' (kilobytes), no 'du -s'
            (blocs)
                espai_usat_kb=$(du -sk "$directori_usuari" 2>/dev/null | cut -f1)
                [ -z "$espai_usat_kb" ] && espai_usat_kb=0 # Gestió d'errors

                total_kb=$((total_kb + espai_usat_kb))
                comprova_usuari "$usuari" "$directori_usuari" "$espai_usat_kb"
            fi
        done

        echo "---"
        # CORRECCIÓ: Formatem el total correctament
        total_hr=$(format_kb_to_hr $total_kb)
        echo "Total Grup $GROUP: $total_hr"

    else
        # --- Mode Tots els Usuaris (Robust) ---

```

```
# CORRECCIÓ: Llegim /etc/passwd (UID >= 1000) en lloc de /home/*

awk -F: '$3 >= 1000 {print $1 ":" $6}' /etc/passwd | while IFS=: read -r
usuari directori; do
    if [ -d "$directori" ]; then
        # CORRECCIÓ CLAU: Utilitzem 'du -sk' (kilobytes)
        espai_usat_kb=$(du -sk "$directori" 2>/dev/null | cut -f1)
        [ -z "$espai_usat_kb" ] && espai_usat_kb=0 # Gestió d'errors

        comprova_usuari "$usuari" "$directori" "$espai_usat_kb"
    fi
done
fi
```

Mostra la sortida de l'execució del script

```
root@davidP (dl. de nov. 10):~# ./ocupacio.sh 600M
homeA          20 KB
homeB          52 KB
root@davidP (dl. de nov. 10):~# ./ocupacio.sh -g 600M
Ús: ./ocupacio.sh [-g grup] max_permès
max_permès ha de ser un valor seguit de K, M o G (per exemple, 600M)
root@davidP (dl. de nov. 10):~# ./ocupacio.sh -g users 600M
Total grup users: 0 MB
```

5.2. Script en Python

Feu el mateix script amb llenguatge Python

```
None
#!/usr/bin/env python3
#
# Script per a la gestió de l'espai en disc (Apartat 5.2)
# Versió refactoritzada, més robusta i segura.
#

import os
import subprocess
```

```

import sys
import grp
import pwd
import argparse # Mòdul recomanat per llegir arguments
import re       # Per processar millor la mida

# Constants per a l'avís al .profile
PROFILE_START_TAG = "#--- AUTO-GENERATED DISK USAGE WARNING (START) ---#"
PROFILE_END_TAG = "#--- AUTO-GENERATED DISK USAGE WARNING (END) ---#"

def convert_size_to_kb(size_str: str) -> int:
    """Converteix una mida (ex: 500M, 1G) a Kilobytes."""
    # Millorem l'original amb una expressió regular per ser més flexible
    size_str = size_str.upper().strip()
    match = re.match(r"^(\d+)([KMG]?)$", size_str)

    if not match:
        # Si no hi ha unitat, assumim bytes (o K, segons la pràctica)
        # Assumirem K, com a l'script original
        if size_str.isdigit():
            return int(size_str)
        raise ValueError(f"Format de mida no reconegut: {size_str}")

    number = int(match.group(1))
    unit = match.group(2)

    units_to_kb = {
        "K": 1,
        "M": 1024,
        "G": 1024**2,
        "": 1 # Si no hi ha unitat, assumim K
    }

    return number * units_to_kb.get(unit, 1)

def format_size_kb(size_kb: int) -> str:
    """Posa la mida en el format més adient (funció original, correcta)."""

```



```

    if size_kb >= 1048576: # >= 1 GB
        return f"{size_kb / 1048576:.2f} GB"
    elif size_kb >= 1024: # >= 1 MB
        return f"{size_kb / 1024:.2f} MB"
    else: # Menys d'1 MB
        return f"{size_kb} KB"

def get_usage_kb(home_dir: str) -> int:
    """
    Retorna l'ús de disc (en KB) d'un directori.
    Retorna 0 si el directori no existeix o hi ha un error.
    """
    if not os.path.isdir(home_dir):
        return 0

    try:
        # Usem subprocess.run (més modern) en lloc de check_output
        result = subprocess.run(
            ['du', '-sk', home_dir],
            capture_output=True, text=True, check=True
        )
        # du -sk retorna: <mida_kb> <path>
        usage_kb = int(result.stdout.split()[0])
        return usage_kb
    except (subprocess.CalledProcessError, FileNotFoundError, ValueError,
            IndexError):
        # Si 'du' falla o el directori no és llegible
        return 0

def update_profile_warning(profile_path: str, user_info: pwd.struct_passwd,
usage_hr: str, limit_hr: str) -> None:
    """
    Actualitza l'avís al fitxer .profile.
    Aquesta funció esborra l'avís antic i n'escriu un de nou.
    """
    lines = []
    in_old_block = False

```

```

try:
    # 1. Llegim el fitxer sencer (si existeix)
    if os.path.exists(profile_path):
        with open(profile_path, 'r') as f:
            for line in f:
                if line.strip() == PROFILE_START_TAG:
                    in_old_block = True
                elif line.strip() == PROFILE_END_TAG:
                    in_old_block = False
                elif not in_old_block:
                    lines.append(line)

    # 2. Afegim el nou avís al final
    lines.append(f"\n{PROFILE_START_TAG}\n")
    lines.append(f"# ATENCIÓ ({user_info.pw_name}): El teu ús de disc
({usage_hr}) supera el límit ({limit_hr}).\n")
    lines.append("# Si us plau, esborra o comprimeix fitxers antics.\n")
    lines.append("# Per esborrar aquest missatge, elimina les línies entre
les etiquetes START i END.\n")
    lines.append(f"{PROFILE_END_TAG}\n")

    # 3. Escrivim el fitxer de nou
    with open(profile_path, 'w') as f:
        f.writelines(lines)

    # 4. Assegurem que el propietari segueixi sent l'usuari
    os.chown(profile_path, user_info.pw_uid, user_info.pw_gid)

except (IOError, OSError) as e:
    print(f"    -> ERROR: No s'ha pogut escriure a {profile_path}: {e}",
file=sys.stderr)

def main():
    # --- 1. Processar Paràmetres (amb argparse) ---
    parser = argparse.ArgumentParser(
        description="Script per a la gestió de l'espai en disc.",

```

```

        usage="% (prog)s [-g grup] max_permès (ex: 500K, 100M)"
    )
    parser.add_argument(
        '-g', '--grup',
        dest='group_name',
        help="Nom del grup a comprovar"
    )
    parser.add_argument(
        'max_limit',
        help="Límit de mida màxim (ex: 600M)"
    )
    args = parser.parse_args()

    try:
        max_permitted_kb = convert_size_to_kb(args.max_limit)
    except ValueError as e:
        print(f"Error: {e}", file=sys.stderr)
        print(f"Ús: {parser.usage}", file=sys.stderr)
        sys.exit(1)

    # --- 2. Obtenir Llista d'Usuaris ---
    users_to_check = []
    if args.group_name:
        # Si s'ha especificat grup
        try:
            # Obtenim la llista de membres del grup
            users_to_check = grp.getgrnam(args.group_name).gr_mem
        except KeyError:
            print(f"El grup '{args.group_name}' no existeix.", file=sys.stderr)
            sys.exit(2)
    else:
        # Si no, agafem tots els usuaris "humans" (UID >= 1000)
        # Això és millor que 'os.listdir("/home")'
        users_to_check = [u.pw_name for u in pwd.getpwall() if u.pw_uid >=
1000]

    # --- 3. Iterar, Comprovar i Avisar ---

```

```

total_kb = 0
print(f"Calculant ús de disc (Límit: {args.max_limit})...")

for user in users_to_check:
    try:
        user_info = pwd.getpwnam(user)
        home_dir = user_info.pw_dir
    except KeyError:
        print(f" -> AVÍS: Usuari '{user}' del grup '{args.group_name}' no trobat. S'ignora.")
        continue

    usage_kb = get_usage_kb(home_dir)
    total_kb += usage_kb
    formatted_usage = format_size_kb(usage_kb)

    # Imprimim l'ús
    print(f"{user}\t\t{formatted_usage}")

    # Comprovem si supera el límit
    if usage_kb > max_permitted_kb:
        print(f" -> AVÍS: {user} supera el límit. S'escriurà a .profile.")
        profile_path = os.path.join(home_dir, '.profile')
        update_profile_warning(profile_path, user_info, formatted_usage,
args.max_limit)

# --- 4. Mostrar Total del Grup ---
if args.group_name:
    print("---")
    print(f"Total Grup {args.group_name}: {format_size_kb(total_kb)}")

if __name__ == "__main__":
    main()

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# python3 ocupacio.py 600M
homeA          20 KB
homeB          52 KB
root@davidP (dl. de nov. 10):~# python3 ocupacio.py -g users 600M
Total grup users: 0 KB

```

6. Informació d'usuaris

Volem fer un script que donat un nom d'usuari ens doni la següent informació relacionada amb ell:

Home

Mida total del directori home (tot incloent subdirectoris)

Directoris fora del directori home on l'usuari te fitxers propis

Nombre de processos actius de l'usuari

Una possible sortida seria:

```

$./infouser.sh aduran
Home: /home/aduran
Home size: 2.5G
Other dirs: /tmp /home/common
Active processes: 5

```

6.1. El script en Bash

Feu el script amb Bash

```

None
#!/bin/bash
#
# Script per mostrar informació d'un usuari (Apartat 6.1)
# Versió corregida i robusta.

if [ $# -ne 1 ]; then
    echo "ús: $0 <nom_d usuari>"
    exit 1
fi

```

```

username="$1" # Guardar nom usuari donat

# --- 1. Home (Mètode segur) ---
# [CORRECCIÓ 4] Fem servir 'getent' en lloc d'eval'
dir_usr=$(getent passwd "$username" | cut -d: -f6)

if [ -z "$dir_usr" ] || [ ! -d "$dir_usr" ]; then
    echo "Usuari $username no existeix o no te una carpeta home"
    exit 1
fi

# --- 2. Mida del Home ---
# Afegim 2>/dev/null per ignorar errors de permís
home_size=$(du -sh "$dir_usr" 2>/dev/null | cut -f1)

# --- 3. Altres Directoris (Lògica corregida) ---
# [CORRECCIÓ 3] Busquem FITXERS (-type f) que pertanyin a l'usuari,
# fora del seu 'home' i de carpetes de sistema sorolloses (/proc, /sys).
# -printf '%h\n' -> imprimeix el directori que conté el fitxer
# sort -u -> mostra només directoris únics
# tr '\n' ' ' -> formata la sortida en una sola línia
other_dirs=$(find / \
    -path "/proc" -prune -o \
    -path "/sys" -prune -o \
    -path "/dev" -prune -o \
    -path "$dir_usr" -prune -o \
    -type f -user "$username" -printf '%h\n' 2>/dev/null \
    | sort -u | tr '\n' ' ')

# --- 4. Processos Actius ---
# [CORRECCIÓ 2] Afegim --no-headers per no comptar la capçalera
active_proc=$(ps -U "$username" --no-headers | wc -l)

# --- 5. Sortida ---
echo "Home: $dir_usr"
echo "Home size: $home_size"
# [CORRECCIÓ 1] Fem servir la variable correcta: $other_dirs

```

```
echo "Other dirs: $other_dirs"
echo "Active processes: $active_proc"
```

Mostra la sortida de l'execució del script

```
root@davidP (dl. de nov. 10):~# nano infousers.sh
root@davidP (dl. de nov. 10):~# chmod +x infousers.sh
root@davidP (dl. de nov. 10):~# ./infousers.sh root
Home:/root
Home size: 245M
Other dirs:
Active processes: 158
```

6.2. Script en Python

Feu el mateix script amb llenguatge Python

```
None
#!/usr/bin/env python3
#
# Script per mostrar informació d'un usuari (Apartat 6.2)
# Versió corregida i robusta.
#

import os
import sys
import pwd
import subprocess # Mòdul modern per executar comandes

usage = "Usage: InfoUsers.py <username>"

# Comprovar arguments (el teu codi original és correcte)
if len(sys.argv) != 2:
    print(usage)
    sys.exit(1)

username = sys.argv[1]
```

```

# Comprovar si l'usuari existeix (el teu codi original és correcte)
try:
    user_info = pwd.getpwnam(username)
except KeyError:
    print(f"User {username} does not exist.")
    sys.exit(1)

# --- 1. Home (Mètode segur) ---
home_dir = user_info.pw_dir # Mètode més segur que os.path.expanduser

if not os.path.isdir(home_dir):
    print(f"User {username} does not have a home directory.")
    sys.exit(1)

# --- 2. Mida del Home ---
# [REFACTOR] Substituïm os.popen per subprocess.run
try:
    # Executem 'du -sh /path/to/home'
    result = subprocess.run(
        ['du', '-sh', home_dir],
        capture_output=True, text=True, check=True, stderr=subprocess.DEVNULL
    )
    home_size = result.stdout.split()[0]
except (subprocess.CalledProcessError, FileNotFoundError) as e:
    home_size = "Error"

# --- 3. Altres Directoris ---
# [REFACTOR] Substituïm os.walk('/') per una comanda 'find' eficient.
# Aquesta comanda és la mateixa que la del script Bash (Apartat 6.1)
# Fa "prune" (poda) de /proc, /sys, /dev i el propi home_dir
# Busca fitxers (-type f) de l'usuari (-user)
# Imprimeix el directori pare (%h) i filtra duplicats (sort -u)
find_cmd = (
    f"find / -path /proc -prune -o -path /sys -prune -o "
    f"-path /dev -prune -o -path '{home_dir}' -prune -o "
    f"-type f -user {username} -printf '%h\\n' 2>/dev/null | sort -u"
)

```



```

try:
    # Necessitem shell=True per interpretar els '-o' (OR) correctament
    result = subprocess.run(
        find_cmd,
        shell=True, capture_output=True, text=True, check=True
    )
    # [CORRECCIÓ 4] Formatem la sortida com una cadena separada per espais
    other_dirs = result.stdout.strip().replace('\n', ' ')
except (subprocess.CalledProcessError, FileNotFoundError):
    other_dirs = "" # Buit si no troba res o hi ha un error

# --- 4. Processos Actius ---
# [REFACTOR] Substituïm os.popen per subprocess.run
try:
    # Executem 'pgrep -u username'
    result = subprocess.run(
        ['pgrep', '-u', username],
        capture_output=True, text=True
    )
    # Comptem quantes línies (PIDs) ha retornat
    if result.stdout.strip():
        active_proc = len(result.stdout.strip().split('\n'))
    else:
        active_proc = 0 # Cap procés
except FileNotFoundError:
    active_proc = "Error (pgrep not found)"

# --- 5. Sortida ---
print(f"Home: {home_dir}")
print(f"Home size: {home_size}")
print(f"Other dirs: {other_dirs}")
print(f"Active processes: {active_proc}")

```

Mostra la sortida de l'execució del script

Per aquesta execució he eliminat els altres directoris, perquè surtien massa i no es veia res a la terminal.

```
root@davidP (dl. de nov. 10):~# nano infousers.py
root@davidP (dl. de nov. 10):~# python3 infousers.py root
Home: /root
Home size: 245M
Active processes: 158
```

7. Estadístiques de login y processos

Volem fer un script (**user-stats**) que ens doni un resum de tots els accessos de tots els usuaris a la màquina. El resum ha d'incloure per a cada usuari del sistema el temps total de login y el nombre total de logins que cada usuari ha fet (veure comanda **last**). A més a més, per als usuaris que tinguin connexions actives es demana reportar el nombre de processos que tenen en execució i el percentatge de CPU que estan utilitzant (veure comanda **ps**).

La sortida de l'script ha de ser similar a :

```
./user-stats.pl

Resum de logins:

Usuari aduran: temps total de login 115 min, nombre total de logins: 10
Usuari marcg: temps total de login 153 min, nombre total de logins: 35


Resum d'usuaris connectats

Usuari alvarez: 3 processos -> 30 % CPU
Usuari root: 10 processos -> 15% CPU
```

7.1. El script en Bash

Feu el script amb Bash

```
None
#!/bin/bash

# Get login summary
login_summary=$(last | awk '{print $1}' | sort | uniq -c | awk '{print "Usuari\n"$2": temps total de login \"$1\" min, nombre total de logins: \"$1\"}')
```

```
# Get active users summary
active_users_summary=$(ps -e -o user,pcpu | awk '{if(NR>1){arr[$1]+=$2;}}
```

```

count[$1++]}} END {for(user in arr){print "Usuari "user": "count[user]" processos
-> "arr[user]/count[user]"% CPU"}}}')

# Print summaries
echo "Resum de logins:"
echo "$login_summary"
echo -e "\nResum d'usuaris connectats"
echo "$active_users_summary"

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# ./user_stats.sh
./user_stats.sh: línia 4: last: no s'ha trobat l'ordre
Resum de logins:

Resum d'usuaris connectats
Usuari root: 160 processos -> 0,01875% CPU
Usuari rtkit: 1 processos -> 0% CPU
Usuari message+: 1 processos -> 0% CPU
Usuari avahi: 2 processos -> 0% CPU
Usuari colord: 1 processos -> 0% CPU
Usuari polkitd: 1 processos -> 0% CPU
Usuari systemd+: 1 processos -> 0% CPU

```

7.2. Script en Python

Feu el mateix script amb llenguatge Python

```

None
#!/usr/bin/env python3
#
# Script 7: Estadístiques de login y processos (user-stats)
#

import subprocess
import pwd
import re
import sys

```

```

def get_login_stats():
    """
    Mostra el resum de logins (temps total i nombre) per a cada usuari
    del sistema, utilitzant la comanda 'last'.
    """
    print("Resum de logins:")

    try:
        # Obtenim tots els usuaris del sistema
        all_users = pwd.getpwall()
    except Exception as e:
        print(f"Error: No s'ha pogut llegir la llista d'usuaris: {e}",
file=sys.stderr)
        return

    for user_entry in all_users:
        user = user_entry.pw_name

        try:
            # Executem 'last -w' per obtenir l'historial complet de l'usuari
            cmd = ['last', '-w', user]
            result = subprocess.run(cmd, capture_output=True, text=True,
check=False, encoding='utf-8')

            if result.returncode != 0 or not result.stdout:
                continue

            lines = result.stdout.strip().splitlines()

            # Filtrem línies que no són de login (com 'wtmp begins...')
            login_lines = [line for line in lines if line and not
line.startswith('wtmp')]

            num_logins = len(login_lines)

            # Si hi ha logins, calculem el temps

```

```

        if num_logins > 0:
            total_min = 0

            # Regex per trobar durades: (HH:MM) o (DD+HH:MM)
            # Grups: (1: Dies), (2: Hores), (3: Minuts)
            duration_regex = re.compile(r'\((?:(\d+)\+)?(\d{2}):(\d{2})\)')

            for line in login_lines:
                match = duration_regex.search(line)
                if match:
                    days_str, hours_str, minutes_str = match.groups()

                    if days_str:
                        total_min += int(days_str) * 1440 # 24 * 60
                    if hours_str:
                        total_min += int(hours_str) * 60
                    if minutes_str:
                        total_min += int(minutes_str)

            print(f"Usuari {user}: temps total de login {total_min} min,
nombre total de logins: {num_logins}")

    except (subprocess.CalledProcessError, FileNotFoundError):
        # Ignorem errors si 'last' falla per a un usuari (ex: usuaris de
sistema)

        pass
    except Exception as e:
        print(f"Error processant logins de {user}: {e}", file=sys.stderr)

def get_active_stats():
    """
    Mostra un resum (processos i %CPU) dels usuaris
    actualment connectats.
    """
    print("\nResum d'usuaris connectats")

    try:

```

```

        # Obtenim la llista d'usuaris connectats amb la comanda 'users'
        result = subprocess.run(['users'], capture_output=True, text=True,
                                check=True, encoding='utf-8')

        # Creem una llista única d'usuaris actius
        active_users = sorted(list(set(result.stdout.strip().split()))

        if not active_users:
            print("No hi ha usuaris connectats actualment.")
            return

        for user in active_users:
            try:
                # Executem 'ps' per obtenir el %CPU de tots els processos de
l'usuari

                cmd = ['ps', '-u', user, '-o', '%cpu', '--no-headers']
                result_ps = subprocess.run(cmd, capture_output=True, text=True,
                                check=True, encoding='utf-8')

                cpu_lines = result_ps.stdout.strip().splitlines()

                num_procs = len(cpu_lines)
                total_cpu = 0.0

                # Sumem el %CPU de cada procés
                for line in cpu_lines:
                    try:
                        total_cpu += float(line.strip())
                    except ValueError:
                        pass # Ignorem línies invàlides

                # Mostrem el resultat amb un decimal
                print(f"Usuari {user}: {num_procs} processos ->
{total_cpu:.1f}% CPU")

            except (subprocess.CalledProcessError, FileNotFoundError):
                print(f"Error: No s'ha pogut executar 'ps' per a l'usuari

```

```

{user}.".", file=sys.stderr)
        except Exception as e:
            print(f"Error processant processos de {user}: {e}",
file=sys.stderr)

        except (subprocess.CalledProcessError, FileNotFoundError):
            print("Error: No s'ha pogut executar la comanda 'users'.",
file=sys.stderr)
        except Exception as e:
            print(f"Error inesperat en obtenir usuaris actius: {e}",
file=sys.stderr)

# --- Execució Principal ---
if __name__ == "__main__":
    get_login_stats()
    get_active_stats()

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# python3 user_stats.py
Resum de logins:

Resum d'usuaris connectats
Usuari root: 159 processos -> 66.6% CPU

```

8. Estadístiques de comunicació

Volem fer un script que ens tregui la següent informació per a cada interfície de xarxa activa:

Nom de la interfície: total de paquets transmesos

Total: suma de tots els paquets transmesos en totes les interfícies actives

Per exemple:

```
$/net-out
```

```
lo:          1621
```

```
wlan0:       64634
```

```
Total: 66255
```

Si ara volem que l'script vagi donant la informació en un terminal cada N segons, com ho faríeu? Supposeu que passem el temps d'espera per paràmetre a l'script, per exemple:

```
$ net-out 2 # amb una espera de 2 segons
```

8.1. El script en Bash

Feu el script amb Bash

None

```
#!/bin/bash

# Guardem els arguments en variables amb noms nous
nom_script="$0"
interval_segons="$1"

# Comprovar si s'ha proporcionat exactament un argument
if [ "$#" -ne 1 ]; then
    echo "Usage: $nom_script <interval_en_segons>"
    exit 1
fi

# Bucle infinit per mostrar les estadístiques periòdicament
while true; do
    # Obténir la llista de noms d'interfícies (ex: eth0, lo)
    llista_interfícies=$(cat /proc/net/dev | grep ':' | awk '{print $1}' | tr
-d ':')

    # Inicialitzar el comptador total de paquets
    suma_paquets=0

    echo "-----"
    # Processar cada interfície de la llista
```



```
for nom_iface in $llista_interficies; do
    # Obtenir els paquets transmesos (columna 10) per a aquesta interfície
    paquets_transmesos=$(cat /proc/net/dev | grep "$nom_iface" | awk
'{print $10}')

    # Acumular al total
    suma_paquets=$((suma_paquets + paquets_transmesos))

    # Mostrar les dades de la interfície
    echo -e "$nom_iface:\t$paquets_transmesos"
done

# Mostrar el total acumulat
echo -e "Total:\t$suma_paquets"
echo "-----"

# Esperar l'interval especificat
sleep "$interval_segons"
done
```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# nano net-out.sh
root@davidP (dl. de nov. 10):~# chmod +x net-out.sh
root@davidP (dl. de nov. 10):~# ./net-out.sh 2
-----
lo:      208360
enp0s3: 13403541
Total:   13611901
-----
-----
lo:      208360
enp0s3: 13403541
Total:   13611901
-----
-----
lo:      208360
enp0s3: 13403541
Total:   13611901
-----
-----
lo:      208360
enp0s3: 13403601
Total:   13611961
-----
-----

```

8.2. Script en Python

Feu el mateix script amb llenguatge Python

```

None
import os
import time
import sys

def recollir_dades_xarxa():
    """Llegeix /proc/net/dev i retorna un diccionari d'interfícies i paquets
    TX."""
    # Obténir les dades de /proc/net/dev
    with open('/proc/net/dev', 'r') as fitxer_proc:
        # 'net_dump' ara es diu 'dades_brutes'
        dades_brutes = fitxer_proc.readlines()

    # Diccionari per emmagatzemar les dades

```

```

# 'interface_data' ara es diu 'resultats_iface'
resultats_iface = {}

# Processar cada línia, ignorant les dues primeres
# 'line' ara es diu 'fila'
for fila in dades_brutes[2:]:
    fila = fila.strip()
    if fila:
        # 'parts' ara es diu 'camps'
        camps = fila.split()
        # 'interface' ara es diu 'nom_iface'
        nom_iface = camps[0].rstrip(':')
        # 'transmit_packets' ara es diu 'comptador_tx'
        # Els paquets transmesos (TX) estan en la desena posició (índex 9)
        comptador_tx = int(camps[9])
        resultats_iface[nom_iface] = comptador_tx

return resultats_iface

def iniciar_monitor(temps_espera):
    """Bucle principal que imprimeix estadístiques cada N segons."""
    # 'main' ara es diu 'iniciar_monitor'
    # 'interval' ara es diu 'temps_espera'
    while True:
        # Obtenir les dades de les interfícies
        # 'stats' ara es diu 'valors_actuais'
        valors_actuais = recollir_dades_xarxa()
        # 'total_packets' ara es diu 'suma_total'
        suma_total = sum(valors_actuais.values())

        # Imprimir les dades
        # 'interface' i 'packets' ara es diuen 'dispositiu' i 'quantitat'
        for dispositiu, quantitat in valors_actuais.items():
            print(f"{dispositiu}: \t{quantitat}")
        print(f"Total:\t{suma_total}\n")

        # Esperar N segons

```

```

        time.sleep(temps_espera)

if __name__ == "__main__":
    # Comprovar si s'ha proporcionat un interval
    if len(sys.argv) != 2:
        # El missatge d'ús es manté igual per claredat
        print("Usage: net-out.py <interval_in_seconds>")
        sys.exit(1)

    # Convertim l'argument a enter
    segons_pausa = int(sys.argv[1])

    # Executar el bucle principal amb l'interval especificat
    # Crida a la funció amb el nou nom
    iniciar_monitor(segons_pausa)

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# nano net-out.py
root@davidP (dl. de nov. 10):~# chmod +x net-out.py
root@davidP (dl. de nov. 10):~# python3 net-out.py 2
lo:      208360
enp0s3:   13434680
Total:   13643040

lo:      208360
enp0s3:   13436108
Total:   13644468

lo:      208360
enp0s3:   13436228
Total:   13644588

lo:      208360
enp0s3:   13436228
Total:   13644588

```

9. Activitat dels usuaris

Volem classificar els usuaris de la màquina que administrem en funció de l'activitat que mostren en el sistema de fitxers. Realitzeu un script `class_act` que donat un nombre enter `n`

i el nom i primer cognom d'un usuari (**atenció, no es tracta del uid**), ens informi del nombre de fitxers en el home de l'usuari, amb data de modificació entre la data actual i els **n o menys dies** anteriors, i l'espai que ocupen a disc.

Exemple:

```
$ ./class_act.sh 3 "Alex Duran"
```

```
Alex Duran (aduran) 150 fitxers modificats que ocupen 1.2 MB
```

9.1. El script en Bash

Feu el script amb Bash

None

```
#!/bin/bash

# Comprova si el nombre d'arguments no és 2
if [ $# -ne 2 ]; then
    echo "Uso: $0 <n> <Nombre y Apellido del usuario>"
    exit 1
fi

# Assigna els arguments a variables amb noms nous
num_dies="$1"
nom_entrat_usuari="$2"

# Inicialitza variables per al directori i les inicials
directori_home=""
lletres_inicials=""

# Obté la línia completa del fitxer passwd
info_passwd="$(getent passwd "$nom_entrat_usuari")"

# Comprova si l'usuari existeix
if [ -z "$info_passwd" ]; then
    echo "El usuario no existe."
    exit 1
else
    # Extreu el 6è camp (directori home)
    directori_home="$(echo "$info_passwd" | cut -d: -f6)"
```

```

    # Extreu el 5è camp (GECOS), i d'aquest, el primer tros separat per espai
    # ATENCIÓ: 'nom_entrat_usuari' es reassigna aquí
    nom_entrat_usuari="$(echo "$info_passwd" | cut -d: -f5 | cut -d' ' -f1)"
    # Obté la primera lletra del nom obtingut
    lletres_inicials="${nom_entrat_usuari:0:1}"
fi

# Calcula la data límit en segons (timestamp)
data_limit_secs="$(date -d "$num_dies days" +%s)"

comptador_fitxers=0 # Declarada como variable global

# Bucle 'find' que envia la sortida a un 'while'
# Aquest 'while' s'executa en un subshell, 'comptador_fitxers' no es
# modificarà fora d'aquest bloc.
find "$directori_home" -type f -print0 | while IFS= read -r -d '$\0'
ruta_completa; do
    # Obté les dades del fitxer (Temps de modificació %Y, Mida %s)
    dades_fitxer="$(stat -c%Y,%s "$ruta_completa" 2>/dev/null)"

    # Si 'stat' falla, salta a la següent iteració
    if [ -z "$dades_fitxer" ]; then
        continue
    fi

    # Extreu el temps i la mida
    temps_modificacio="$(echo "$dades_fitxer" | cut -d, -f1)"
    mida_bytes_fitxer="$(echo "$dades_fitxer" | cut -d, -f2)" # Aquesta
variable no s'utilitza

    # Compara el temps del fitxer amb el límit
    if [ "$temps_modificacio" -ge "$data_limit_secs" ]; then
        comptador_fitxers=$((comptador_fitxers+1)) # Modificamos la variable
global (dins del subshell)
    fi
done

# Aquesta variable 'mida_total_bytes' s'inicialitza a 0 aquí baix

```

```
# i no acumula la 'mida_bytes_fitxer' de dins del bucle.
mida_total_bytes=0 # Otras variables locales pueden ser declaradas dentro del
bucle

# El càlcul de MB es farà sobre 0
mida_total_mb="$(bc -l <<< "scale=1; $mida_total_bytes / (1024 * 1024)")"

# Imprimeix el resultat. 'comptador_fitxers' serà 0 a causa del subshell.
echo "${nom_entrat_usuari} (${lletres_inicials}) ${comptador_fitxers} archivos
modificados que ocupan ${mida_total_mb} MB"
```

Mostra la sortida de l'execució del script

```
root@davidP (dl. de nov. 10):~# ./class_act.sh 1 "root"
root (r) 0 archivos modificados que ocupan 0 MB
```

9.2. Script en Python

Feu el mateix script amb llenguatge Python

```
None

import os
import sys
import datetime
import pwd

def generar_informe_activitat(periode_dies, compte_usuari):
    """
    Analitza l'activitat de fitxers recents d'un usuari.
    """
    try:
        # 'user_info' ara es diu 'info_compte'
        info_compte = pwd.getpwnam(compte_usuari)
    except KeyError:
        print("L'usuari no existeix.")
        return
```

```

# 'user_home' ara es diu 'directori_personal'
directori_personal = info_compte.pw_dir

# 'user_initials' ara es diu 'inicial_nom'
inicial_nom = compte_usuari.split()[0][0].lower()

# Calcular la data límit
# 'limit_date' ara es diu 'timestamp_limit'
timestamp_limit = (datetime.datetime.now() -
datetime.timedelta(days=periode_dies)).timestamp()

# 'file_count' ara es diu 'comptador_arxius'
comptador_arxius = 0
# 'total_size' ara es diu 'mida_total_bytes'
mida_total_bytes = 0

# 'root', 'dirs', 'files' ara es diuen 'directori_actual', 'subdirectoris',
'llista_arxius'
for directori_actual, subdirectoris, llista_arxius in
os.walk(directori_personal):
    # 'file' ara es diu 'nom_arxiu'
    for nom_arxiu in llista_arxius:
        # 'file_path' ara es diu 'ruta_completa'
        ruta_completa = os.path.join(directori_actual, nom_arxiu)
        try:
            # 'file_stat' ara es diu 'metadades_arxiu'
            metadades_arxiu = os.stat(ruta_completa)
        except FileNotFoundError:
            continue # Ignorar fitxers que no existeixen

# 'file_modification_time' ara es diu 'temps_modificacio'
temps_modificacio = metadades_arxiu.st_mtime

if temps_modificacio >= timestamp_limit:
    comptador_arxius += 1
    mida_total_bytes += metadades_arxiu.st_size

# 'total_size_mb' ara es diu 'mida_final_mb'

```



```

mida_final_mb = mida_total_bytes / (1024 * 1024)

# La impressió utilitza les noves variables
    print(f"{compte_usuari}  ({inicial_nom})  {comptador_arxius}  fitxers
modificats que ocupen {mida_final_mb:.1f} MB")

if __name__ == "__main__":
    if len(sys.argv) != 3:
        # El missatge d'ús es manté igual per claredat
        print("Ús: python class_act.py <n> <Nom i Cognom de l'usuari>")
        sys.exit(1)

    # 'n' ara es diu 'dies_a_revisar'
    dies_a_revisar = int(sys.argv[1])
    # 'user_name' ara es diu 'nom_usuari_argument'
    nom_usuari_argument = sys.argv[2]

    # Crida a la funció amb els noms nous
    generar_informe_activitat(dies_a_revisar, nom_usuari_argument)

```

Mostra la sortida de l'execució del script

```

root@davidP (dl. de nov. 10):~# python3 class_act.py 1 "root"
root (r) 1150 fitxers modificats que ocupen 91.5 MB

```

10. Referències Bibliogràfiques

[1] M. Garrels. **Bash Guide for Beginners**. Online: The Linux Documentation Project.

<http://tldp.org/LDP/Bash-Beginners-Guide/Bash-Beginners-Guide.pdf>

[2] D. Robbins, **Bash by example**. Online: IBM Developer Works

<http://www-128.ibm.com/developerworks/linux/library/l-bash.html?ca=drs->

[3] Python Software foundation

<https://www.python.org/>

[4] **The Python Tutorial**

<https://docs.python.org/3/tutorial/index.html>

[5] PyCharm Edu. Easy and Professional Tool to Learn & Teach Programming with Python

<https://www.jetbrains.com/pycharm-edu/>